

Lab 5: Data Confidentiality in Information Security

1. Objective:

To understand and implement the concept of confidentiality by using symmetric encryption to protect sensitive information such as passwords or data.

2. Tools & Req:

- Python 3
- Google Colab
- cryptography library (install using `pip install cryptography`)
- Any text editor or IDE (VS Code, PyCharm, Jupyter Notebook)

3. Setup Environment

1. Open your Python / Collab Terminal.
2. Install the required library:
`pip install cryptography`

4a. Python Code

(Single Message Encryption)

Generate a secret key (keep this private)

```
key = Fernet.generate_key()
cipher = Fernet(key)
print("Secret Key:", key.decode())
```

Encrypt a confidential message

```
message = input("Enter confidential message: ").encode()
encrypted_message = cipher.encrypt(message)
print("\nEncrypted Message:", encrypted_message)
```

Decrypt the message (verify confidentiality)

```
decrypted_message = cipher.decrypt(encrypted_message)
print("\nDecrypted Message:",
decrypted_message.decode())
```

4b. Python Code

(Batch Messages Encryption)

Imports the Fernet class from the cryptography library.

```
from cryptography.fernet import Fernet
```

Generates a new random symmetric key (a URL-safe Base64-encoded 32-byte value).

```
key = Fernet.generate_key()
```

Creates a Fernet cipher object using the key.

```
cipher = Fernet(key)
```

A Python list of three plaintext strings (simulating confidential patient records).

```
messages = [
```

```
"Patient ID 101: Diabetes report confidential",
```

```
"Patient ID 102: COVID-19 test result confidential",
```

```
"Patient ID 103: Prescription data confidential"
```

```
]
```

```
encrypted_messages = [cipher.encrypt(m.encode()) for m in messages]
```

Prints the encrypted tokens.

```
print("\nEncrypted Batch:")
```

e is a bytes Fernet token; .decode() converts it to a str so it prints nicely. The printed tokens look like long Base64 strings (e.g. gAAAAAB...==)

```
for e in encrypted_messages: print(e.decode())
```

```
print("\nDecrypted Batch:")
```

Decrypts each token and prints the original plaintext:

```
for e in encrypted_messages: print(cipher.decrypt(e).decode())
```

4c. Write the Python Code that Ask Secret Key to Decrypt

```
from cryptography.fernet import Fernet

# Step 1: Generate a secret key (keep it private and safe)
key = Fernet.generate_key()
cipher = Fernet(key)
print("Secret Key:", key.decode())

# Step 2: Get confidential message from user
message = input("\nEnter confidential message: ").encode()

# Step 3: Encrypt the message
encrypted_message = cipher.encrypt(message)
print("\nEncrypted Message:", encrypted_message.decode())

# Step 4: Simulate sending ciphertext (no key sharing)
print("\n--- Decryption Phase ---")
user_key = input("Enter the secret key to decrypt: ").encode()

try:

    # Step 5: Use user-entered key to create cipher object
    decrypt_cipher = Fernet(user_key)

    # Step 6: Attempt decryption
    decrypted_message = decrypt_cipher.decrypt(encrypted_message)
    print("\nDecrypted Message:", decrypted_message.decode())
    print("\nData confidentiality verified successfully!")

except Exception as e:
    print("\nDecryption failed! Invalid or incorrect key provided.")
    print(f"Error details: {e}")
```

Lab Task:

Practice the code and attach screenshots.